

printquote

Presupuesta una impresión 3D en segundos: arrastra un STL, míralo en 3D y obtén peso, tiempo estimado y precio. Tu archivo no sale de tu navegador.

Repositorio	https://github.com/BertMarti/printquote
Web publicada	https://bertmarti.github.io/printquote/
Stack	TypeScript estricto · Vite · three.js · Vitest · ESLint · GitHub Actions · GitHub Pages
Fecha	30 de septiembre de 2026
Autoría	Alberto Martínez, con un equipo de agentes de IA (Claude Code + OpenCode)

Contenido

1. Qué es y para quién
 2. Funcionamiento a nivel de usuario
 3. Funcionamiento a nivel técnico
 4. Flujo de un presupuesto
 5. Funcionalidades
 6. Calidad, CI/CD y equipo
 7. Cómo explotarlo económicamente
 8. Sigüientes pasos y acciones necesarias por ti, Alberto
- Anexo: código fuente

Qué es y para quién

printquote es una web que calcula el presupuesto de una pieza de impresión 3D. Arrastras un archivo STL, lo ves en 3D sobre la cama de la impresora y obtienes al instante su volumen, peso, tiempo estimado y precio final con el desglose de material, energía y margen.

Todo ocurre en el navegador: no hay servidor, ni cuentas, ni analítica. El archivo nunca se sube a ningún sitio.

Perfil	Para qué lo usa
Maker con impresora	Saber cuánto le cuesta de verdad una pieza antes de imprimirla o de venderla.
Pequeño negocio de impresión	Dar presupuestos rápidos y coherentes a sus clientes, con margen incluido.
Cliente final	Hacerse una idea del coste de un modelo descargado antes de encargarlo.
Docente o estudiante	Entender la relación entre geometría, relleno, material y coste.

Funcionamiento a nivel de usuario

The screenshot shows the printquote web interface. On the left, a 3D model of a piece is displayed on a grid. The grid has axes labeled X, Y, and Z. The piece is a simple, angular shape. On the right, there is a sidebar with technical specifications and pricing. The sidebar is divided into sections: 01 PIEZA, 02 MATERIAL, and 03 IMPRESIÓN. The 01 PIEZA section shows the volume (123,83 cm³), surface (320,78 cm²), dimensions (70,0 x 90,0 x 88,0 mm), and number of triangles (56). The 02 MATERIAL section shows a table with columns for material (PLA, PETG, ABS, TPU) and price (1,24, 1,27, 1,04, 1,21). The 03 IMPRESIÓN section shows the fill percentage (20%), perimeter (2 uds.), line width (0,45 mm), and volumetric flow (8 mm³/s). At the bottom, the total price is displayed as 1,58 €.

01 ■ PIEZA	
Volumen	123,83 cm³
Superficie	320,78 cm²
Dimensiones X x Y x Z	70,0 x 90,0 x 88,0 mm
Triángulos	56

02 ■ MATERIAL	
PLA	1,24
PETG	1,27
ABS	1,04
TPU	1,21

DENSIDAD EN G/CM³

Precio del material 20 €/kg

03 ■ IMPRESIÓN	
Relleno	20 %
Perímetros	2 uds.
Ancho de línea	0,45 mm
Caudal volumétrico	8 mm³/s

Cama X 220 Y 220 Z 250 mm

TOTAL 1,58 €

1 copia - PLA 20 % - 1 h 45 min (estimación)

Copiar presupuesto Imprimir

printquote con la pieza de ejemplo: visor 3D y ficha técnica con el presupuesto en vivo.

1. Abre <https://bertmarti.github.io/printquote/>. Para verlo con una pieza ya cargada: [.../printquote/#ejemplo](https://bertmarti.github.io/printquote/#ejemplo).
2. Arrastra un STL a cualquier parte de la ventana o pulsa **Abrir STL**. Si no tienes ninguno, pulsa **Probar con pieza de ejemplo**.
3. Gira la pieza arrastrando, acércala con la rueda y desplázala con el botón derecho. **Restablecer vista** la vuelve a encuadrar.
4. En la ficha de la derecha revisa la sección **01 Pieza**: volumen, superficie, dimensiones y triángulos. Si la malla está abierta o no cabe en la cama, aparece un aviso.

5. Elige el **material** (PLA, PETG, ABS o TPU) y ajusta su precio por kilo.
6. Ajusta la **impresión**: relleno, perímetros, ancho de línea, caudal y tamaño de la cama.
7. Ajusta **energía y negocio**: potencia media, precio del kWh, margen y número de copias.
8. El **total** se actualiza en vivo en el bloque inferior, con el tiempo rotulado como estimación.
9. Pulsa **Copiar presupuesto** para pegarlo en un mensaje o **Imprimir** para obtener una hoja limpia (o un PDF) con la vista 3D y el desglose.

Los ajustes se recuerdan en tu navegador para la próxima vez. La web cambia sola a modo oscuro si tu sistema lo usa, y se puede manejar entera con el teclado (↑/↓ ajustan los números; con Mayús, de 10 en 10).

Funcionamiento a nivel técnico

Arquitectura

Módulo	Responsabilidad
src/stl/parse.ts	Parser STL propio. Distingue binario y ASCII por tamaño (84 + 50·n bytes) y contenido, no solo por la cabecera «solid». Errores claros en español.
src/stl/analyze.ts · analyzer.ts · stl.worker.ts	Lectura y análisis en un Web Worker para no bloquear la interfaz con archivos grandes; si el worker falla, se hace en el hilo principal. Descarta resultados viejos si se suelta otro archivo.
src/stl/geometry.ts	Volumen (suma de tetraedros con signo), área, caja envolvente, triángulos, aristas abiertas y normales invertidas.
src/quote/	Modelo de coste como funciones puras: materiales, cáscara, relleno, peso, tiempo, energía, margen y copias; formato es-ES; texto del presupuesto.
src/viewer/viewer.ts	Escena three.js: cama con rejilla de 10 mm, cámara con OrbitControls, luces sobrias, material mate y aristas.
src/ui/	Arranque de la app, campos numéricos accesibles, almacenamiento en localStorage y hoja de impresión.
scripts/generate-sample.mjs	Genera la pieza de ejemplo original (soporte de móvil low-poly) como STL binario.
src/viewer/webgl.ts	Carga del visor three.js bajo demanda (import dinámico): el JavaScript inicial pasa de 580 kB a 26 kB.

Cómo se calcula

grosor de pared = perímetros × ancho de línea
cáscara = mín(área × grosor de pared, volumen)
relleno = relleno% × (volumen - cáscara)
volumen impreso = cáscara + relleno (por copia)

peso (g) = volumen impreso (cm³) × densidad (g/cm³)
coste material = peso (kg) × precio (€/kg)
tiempo (h) = volumen impreso / caudal (mm³/s) + 5 min ← estimación
energía (kWh) = potencia media (W) × tiempo (h) / 1000
coste energía = energía × precio (€/kWh)

subtotal = (coste material + coste energía) × copias
TOTAL = subtotal × (1 + margen%)

Ejemplo comprobado en los tests: un cubo de 20 mm tiene 8.000 mm³ de volumen y 2.400 mm² de superficie. En PLA con los valores por defecto sale una cáscara de 2.160 mm³ más 1.168 mm³ de relleno: 4,13 g, 12 minutos estimados y 0,10 € en total (cada importe se redondea a céntimos y el total es la suma de las líneas, como en una factura).

Límites honestos: no se tienen en cuenta soportes, balsa, purga ni altura de capa, y la cáscara se aproxima como área × grosor. El tiempo es siempre una estimación: la cifra real la da el laminador.

Decisiones de diseño

- **Sin servidor:** coste cero de alojamiento (GitHub Pages) y privacidad total.
- **Parser propio** en lugar de un cargador genérico: permite errores claros, detección robusta del formato y tests con STL contruidos en memoria.
- **Campos de texto con inputmode decimal** en lugar de type=number, para aceptar coma o punto decimal igual en todos los navegadores.

- **El naranja nunca como texto sobre fondo claro** (contraste 2,85:1): se usa en rellenos y en la cifra del total sobre fondo tinta, para cumplir AA.
- **base: /printquote/** también en desarrollo, para que el servidor local y el de vista previa se comporten igual que Pages.

Flujo de un presupuesto



Funcionalidades

Funcionalidad	Detalle
Carga de STL	Binario y ASCII; arrastrar y soltar o selector de archivos.
Pieza de ejemplo	Soporte de móvil low-poly original; enlace directo con #ejemplo.
Mediciones	Volumen, superficie, dimensiones X × Y × Z y número de triángulos.
Avisos	Malla abierta, normales invertidas y pieza que no cabe en la cama (permite girar 90° sobre Z).
Visor 3D	three.js con rejilla de cama, órbita, zoom, desplazamiento y restablecer vista; modo oscuro.
Materiales	PLA, PETG, ABS y TPU con densidad y precio por kilo editable.
Parámetros de impresión	Relleno, perímetros, ancho de línea, caudal volumétrico y tamaño de cama.
Energía y negocio	Potencia media, precio del kWh, margen y copias.
Presupuesto en vivo	Desglose y total recalculados al instante; tiempo rotulado como estimación.
Copiar e imprimir	Presupuesto en texto plano y hoja de impresión limpia con la vista 3D.
Memoria de ajustes	localStorage del navegador, con manejo de errores.
Accesibilidad	Etiquetas, foco visible, teclado completo, contraste AA, diseño adaptable a móvil.

Calidad, CI/CD y equipo

- **Tests (Vitest):** parser binario y ASCII con buffers en memoria, geometría (cubo de 20 mm: 8.000 mm³ y 2.400 mm²), archivos corruptos, pieza de ejemplo cerrada y modelo de coste (relleno 0 % y 100 %, copias y margen).
- **CI** (ci.yml): en cada PR y en main, en Ubuntu y Windows: npm ci, lint, tests y build.
- **CD** (deploy.yml): al actualizar main, construye y publica en GitHub Pages.
- **Git:** main protegida (solo por PR), commits convencionales y una rama por agente: agent/builder, agent/qa y agent/opensource-docs.
- **Contexto para IA:** AGENTS.md, CLAUDE.md, opencode.json y MEMORY.md, que cada agente actualiza al terminar.
- **Revisión QA** (Claude Opus, 132 tests): STL con BOM, nombres de sólido con palabras clave, números fuera de float32, binarios con recuento 0, ASCII grandes (de ~800 MB de memoria a un tokenizador byte a byte), soldado de vértices con tolerancia, Web Worker, visor bajo demanda, desglose que no sumaba el total, avisos accesibles con regiones vivas, Open Graph y aviso de pieza en metros o pulgadas.
- **Documentación** (Claude Sonnet): docs/USO.md con 14 secciones, ejemplos de cálculo comprobados con la pieza de ejemplo y guía de calibración con el laminador, y CONTRIBUTING.md.
- **Pull requests:** #1 builder, #2 qa (sobre builder) y #3 docs (sobre qa), todos con el CI en verde.

Cómo explotarlo económicamente

El nicho de la impresión 3D tiene mucha gente que vende piezas y poca herramienta sencilla y bonita para presupuestar. Ideas ordenadas de menor a mayor esfuerzo:

Idea	Cómo	Esfuerzo
Afiliación de filamento	Enlaces de afiliado a cada material (Amazon, tiendas de filamento) en el selector de material.	Bajo
Perfiles de impresora	Perfiles precargados (Bambu, Prusa, Creality...) con caudal y potencia reales. Gratis los básicos, pack premium.	Bajo
Presupuesto con tu marca	Versión Pro para negocios: logotipo, datos fiscales, IVA, numeración y PDF de presupuesto. Pago único o suscripción baja.	Medio
Widget para tiendas	Un fragmento que las tiendas de impresión 3D pegan en su web para que sus clientes suban un STL y vean el precio.	Medio
Marketplace de encargos	Conectar a quien tiene un STL con makers cercanos que lo imprimen, cobrando comisión.	Alto
Integración con laminadores	Leer G-code o 3MF para dar el tiempo y el filamento reales en lugar de estimaciones.	Medio

El primer paso barato: publicar la herramienta en comunidades de impresión 3D (Reddit r/3Dprinting, foros en español, grupos de Telegram), medir visitas y preguntar qué echan en falta.

Siguientes pasos y acciones necesarias por ti, Alberto

Acciones necesarias por ti

1. **Fusionar los PR en orden** (builder → qa → docs). El sistema de permisos no me deja fusionar en main. Al fusionar, se publica la web automáticamente.
2. Comprobar la web publicada: <https://bertmarti.github.io/printquote/#ejemplo>
3. Probar con STL reales tuyos y comparar el tiempo y el peso con lo que dice tu laminador para ajustar los valores por defecto (caudal y sobrecarga).

Mejoras técnicas propuestas

- Soporte de 3MF y OBJ además de STL.
- Estimación de soportes según voladizos y orientación automática óptima.
- Varios objetos a la vez y presupuesto por lote.
- Traducción al inglés para ampliar audiencia.

Anexo: código fuente

Archivos principales tal y como están en main al generar este documento. Las líneas muy largas se parten y continúan en la línea siguiente.

package.json · 37 líneas

```
{
  "name": "printquote",
  "private": true,
  "version": "0.1.0",
  "description": "Presupuesta una impresión 3D en el navegador a partir de un STL.",
  "type": "module",
  "license": "MIT",
  "engines": {
    "node": ">=22.12"
  },
  "scripts": {
    "dev": "vite",
    "build": "npm run typecheck && vite build",
    "preview": "vite preview",
    "lint": "eslint .",
    "test": "vitest run",
    "test:watch": "vitest",
    "sample": "node scripts/generate-sample.mjs",
    "typecheck": "tsc --noEmit && tsc --noEmit -p tests"
  },
  "dependencies": {
    "three": "^0.186.1"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
    "@types/node": "^22.20.4",
    "@types/three": "^0.186.0",
    "eslint": "^10.11.0",
    "globals": "^17.12.0",
    "happy-dom": "^20.14.5",
    "typescript": "^6.0.3",
    "typescript-eslint": "^8.71.0",
    "vite": "^8.3.1",
    "vitest": "^5.0.2"
  }
}
```

vite.config.ts · 18 líneas

```
import { defineConfig } from 'vitest/config';

// La web se publica en GitHub Pages bajo /printquote/. Se usa la misma base en
// desarrollo, build y preview para que las rutas se comporten igual en todas partes.
export default defineConfig({
  base: '/printquote/',
  build: {
    target: 'es2022',
    // El chunk del visor (three.js) pesa ~560 kB sin comprimir; es esperado. Se carga
    // con import() después de pintar la interfaz, así que no retrasa el primer render.
    chunkSizeWarningLimit: 800,
  },
  test: {
    include: ['tests/**/*.test.ts'],
    environment: 'node',
  },
});
```

src/main.ts · 5 líneas

```
import './styles.css';
import { startApp } from './ui/app';

startApp();
```

src/stl/types.ts · 38 líneas

```
/** Formato de origen del archivo STL. */
export type StlFormat = 'binary' | 'ascii';

/**
 * Malla triangular sin indexar tal como viene en el STL.
 * `positions` guarda 9 números por triángulo (x, y, z de sus tres vértices), en mm.
 */
export interface Mesh {
  readonly positions: Float32Array;
  readonly triangleCount: number;
  readonly format: StlFormat;
}

export interface Vec3 {
  readonly x: number;
  readonly y: number;
  readonly z: number;
}

export interface BoundingBox {
  readonly min: Vec3;
  readonly max: Vec3;
  readonly size: Vec3;
}

export interface MeshStats {
  readonly triangleCount: number;
  /** Volumen en mm³ (valor absoluto). */
  readonly volume: number;
  /** Volumen con signo antes del valor absoluto. Negativo o ≈ 0 indica malla abierta o invertida. */
  readonly signedVolume: number;
  /** Área de superficie en mm². */
  readonly surfaceArea: number;
  readonly bounds: BoundingBox;
  /** Aristas que no comparten exactamente dos triángulos (bordes abiertos o no variedad). */
  readonly openEdges: number;
}
```

src/stl/parse.ts · 245 líneas

```
import type { Mesh } from './types';

/** Error de lectura de STL con un mensaje pensado para mostrarse tal cual a la persona usuaria. */
export class StlParseError extends Error {
  constructor(message: string) {
    super(message);
    this.name = 'StlParseError';
  }
}

const HEADER_BYTES = 80;
const COUNT_BYTES = 4;
const BINARY_PREAMBLE = HEADER_BYTES + COUNT_BYTES;
const BYTES_PER_TRIANGLE = 50;
/** Tamaño de la muestra que se inspecciona para decidir si el archivo es texto. */
const TEXT_SNIFF_BYTES = 1024;
/** Marca de orden de bytes de UTF-8 que algunos editores de Windows añaden al principio. */
const UTF8_BOM = [0xef, 0xbb, 0xbf] as const;
/** Una coordenada ASCII más larga que esto no es un número razonable. */
const MAX_NUMBER_TOKEN = 64;

/**
 * Lee un STL binario o ASCII.
 *
 * La detección no se fía solo de la cabecera «solid»: muchos exportadores escriben
 * «solid ...» en la cabecera de archivos binarios. Se decide así:
 * 1. Si el tamaño coincide exactamente con 84 + 50 × n° de triángulos, es binario,
 *    salvo que empiece por «solid», sea texto y contenga «facet» (ASCII casual).
 * 2. Si parece texto y empieza por «solid», es ASCII.
 * 3. Si el tamaño es mayor de lo esperado (relleno al final), se acepta como binario.
 * 4. Si el recuento es 0 pero el tamaño cuadra con 84 + 50 × n, se leen esos n
 *    triángulos (hay exportadores que no rellenan el recuento).
 */
export function parseStl(data: ArrayBuffer | Uint8Array): Mesh {
  const bytes = data instanceof Uint8Array ? data : new Uint8Array(data);

  if (bytes.byteLength === 0) {
    throw new StlParseError('El archivo está vacío.');
```

```
  }

  const looksAscii = startsWithSolid(bytes) && isMostlyText(bytes);

  if (bytes.byteLength >= BINARY_PREAMBLE) {
    const view = new DataView(bytes.buffer, bytes.byteOffset, bytes.byteLength);
    const count = view.getUint32(HEADER_BYTES, true);
    const expected = BINARY_PREAMBLE + count * BYTES_PER_TRIANGLE;

    if (expected === bytes.byteLength && !(looksAscii && containsFacet(bytes))) {
      return parseBinary(view, count);
    }
    if (looksAscii) {
      return parseAscii(bytes);
    }
    if (count > 0 && expected < bytes.byteLength) {
      return parseBinary(view, count);
    }
    const inferred = (bytes.byteLength - BINARY_PREAMBLE) / BYTES_PER_TRIANGLE;
    if (count === 0 && Number.isInteger(inferred)) {
      return parseBinary(view, inferred);
    }
    throw new StlParseError(
      'No parece un STL válido: el tamaño del archivo no coincide con el número de triángulos que declara y tampoco es un STL de texto.',
    );
  }

  if (looksAscii) {
    return parseAscii(bytes);
  }
  throw new StlParseError('No parece un STL válido: el archivo es demasiado pequeño.');
```

```
  }

  function parseBinary(view: DataView, count: number): Mesh {
    if (count === 0) {
      throw new StlParseError('El STL no contiene ningún triángulo.');
```

```

const positions = new Float32Array(count * 9);
for (let t = 0; t < count; t++) {
  // Cada triángulo: normal (12 bytes), 3 vértices (36 bytes), atributo (2 bytes).
  const base = BINARY_PREAMBLE + t * BYTES_PER_TRIANGLE + 12;
  for (let k = 0; k < 9; k++) {
    const value = view.getFloat32(base + k * 4, true);
    if (!Number.isFinite(value)) {
      throw new StlParseError(`El triángulo ${t + 1} tiene coordenadas no válidas.`);
    }
    positions[t * 9 + k] = value;
  }
}
return { positions, triangleCount: count, format: 'binary' };
}

/**
 * Recorre el texto byte a byte, sin decodificarlo entero ni partirlo en un array de
 * palabras: con archivos de cientos de MB eso multiplicaba la memoria por diez.
 */
class Tokenizer {
  pos: number;
  start = 0;
  end = 0;

  constructor(private readonly bytes: Uint8Array, from: number) {
    this.pos = from;
  }

  /** Avanza a la siguiente palabra. Cualquier byte ≤ 0x20 (espacio, tabulador, CR, LF...) separa. */
  next(): boolean {
    const { bytes } = this;
    let i = this.pos;
    while (i < bytes.length && (bytes[i] ?? 0) <= 0x20) i++;
    if (i >= bytes.length) {
      this.pos = i;
      return false;
    }
    this.start = i;
    while (i < bytes.length && (bytes[i] ?? 0) > 0x20) i++;
    this.end = this.pos = i;
    return true;
  }

  /** ¿La palabra actual es `word` (en minúsculas), sin distinguir mayúsculas? */
  is(word: string): boolean {
    if (this.end - this.start !== word.length) return false;
    for (let k = 0; k < word.length; k++) {
      // `| 0x20` pasa A-Z a minúsculas; `word` solo contiene letras minúsculas.
      if (((this.bytes[this.start + k] ?? 0) | 0x20) !== word.charCodeAt(k)) return false;
    }
    return true;
  }

  /** La palabra actual como número; NaN si no tiene forma de número decimal (nada de «0x10» ni «inf»). */
  number(): number {
    if (this.end - this.start > MAX_NUMBER_TOKEN) return Number.NaN;
    let text = '';
    for (let i = this.start; i < this.end; i++) {
      const b = this.bytes[i] ?? 0;
      const isNumberChar = (b >= 0x30 && b <= 0x39) || b === 0x2e || b === 0x2b || b === 0x2d || b === 0x65 || b ===
0x45;
      if (!isNumberChar) return Number.NaN;
      text += String.fromCharCode(b);
    }
    return Number(text);
  }

  /** Salta el resto de la línea (el nombre tras «solid»/«endsolid»), salvo que no haya más saltos de línea. */
  skipLine(): void {
    const { bytes } = this;
    for (let i = this.pos; i < bytes.length; i++) {
      if (bytes[i] === 0x0a || bytes[i] === 0x0d) {
        this.pos = i;
        return;
      }
    }
  }
}
// Archivo en una sola línea: no se puede saber dónde acaba el nombre; se sigue leyendo.

```

```

    }
}

function parseAscii(bytes: Uint8Array): Mesh {
    const tokens = new Tokenizer(bytes, hasBom(bytes) ? UTF8_BOM.length : 0);
    let values = new Float32Array(9 * 1024);
    let length = 0;
    let facet = 0;
    let verticesInFacet = 0;
    let insideFacet = false;

    while (tokens.next()) {
        if (tokens.is('vertex')) {
            if (!insideFacet) {
                throw new StlParseError('Hay un vértice fuera de una cara («facet»).');
            }
            if (verticesInFacet === 3) {
                throw new StlParseError('La cara ${facet} no tiene exactamente 3 vértices.');
```

```

}

/** Texto = sin bytes nulos ni de control (salvo espacios en blanco) en la muestra inicial. */
function isMostlyText(bytes: Uint8Array): boolean {
  const end = Math.min(bytes.length, TEXT_SNIFF_BYTES);
  for (let i = 0; i < end; i++) {
    const b = bytes[i] ?? 0;
    if (b < 0x20 && !isWhitespace(b)) return false;
  }
  return true;
}

function containsFacet(bytes: Uint8Array): boolean {
  const sample = String.fromCharCode(...bytes.subarray(0, TEXT_SNIFF_BYTES)).toLowerCase();
  return sample.includes('facet');
}

```

src/stl/geometry.ts · 222 líneas

```
import type { BoundingBox, Mesh, MeshStats, Vec3 } from './types';

/** Tamaño de cama por defecto (mm): ancho X, fondo Y, alto Z. */
export const DEFAULT_BED: Vec3 = { x: 220, y: 220, z: 250 };

/** Umbral relativo por debajo del cual el volumen se considera = 0. */
const RELATIVE_VOLUME_EPSILON = 1e-6;

export function computeBounds(positions: Float32Array): BoundingBox {
  if (positions.length === 0) {
    const zero = { x: 0, y: 0, z: 0 };
    return { min: zero, max: zero, size: zero };
  }
  let minX = Infinity, minY = Infinity, minZ = Infinity;
  let maxX = -Infinity, maxY = -Infinity, maxZ = -Infinity;
  for (let i = 0; i < positions.length; i += 3) {
    const x = positions[i] ?? 0, y = positions[i + 1] ?? 0, z = positions[i + 2] ?? 0;
    if (x < minX) minX = x;
    if (y < minY) minY = y;
    if (z < minZ) minZ = z;
    if (x > maxX) maxX = x;
    if (y > maxY) maxY = y;
    if (z > maxZ) maxZ = z;
  }
  return {
    min: { x: minX, y: minY, z: minZ },
    max: { x: maxX, y: maxY, z: maxZ },
    size: { x: maxX - minX, y: maxY - minY, z: maxZ - minZ },
  };
}

/**
 * Calcula volumen, área, caja envolvente y aristas abiertas.
 *
 * Volumen: suma de los volúmenes con signo de los tetraedros que forma cada
 * triángulo con un punto de referencia ( $v1 \cdot (v2 \times v3) / 6$ ). Se usa el centro de
 * la caja como referencia para no perder precisión con piezas lejos del origen.
 */
export function computeStats(mesh: Mesh): MeshStats {
  const p = mesh.positions;
  const bounds = computeBounds(p);
  const cx = (bounds.min.x + bounds.max.x) / 2;
  const cy = (bounds.min.y + bounds.max.y) / 2;
  const cz = (bounds.min.z + bounds.max.z) / 2;

  let signedVolume6 = 0;
  let area2 = 0;

  for (let i = 0; i + 8 < p.length; i += 9) {
    const ax = (p[i] ?? 0) - cx, ay = (p[i + 1] ?? 0) - cy, az = (p[i + 2] ?? 0) - cz;
    const bx = (p[i + 3] ?? 0) - cx, by = (p[i + 4] ?? 0) - cy, bz = (p[i + 5] ?? 0) - cz;
    const qx = (p[i + 6] ?? 0) - cx, qy = (p[i + 7] ?? 0) - cy, qz = (p[i + 8] ?? 0) - cz;

    // Volumen con signo:  $a \cdot (b \times c)$ 
    signedVolume6 += ax * (by * qz - bz * qy) + ay * (bz * qx - bx * qz) + az * (bx * qy - by * qx);

    // Área:  $|(b - a) \times (c - a)| / 2$ 
    const ux = bx - ax, uy = by - ay, uz = bz - az;
    const vx = qx - ax, vy = qy - ay, vz = qz - az;
    const nx = uy * vz - uz * vy, ny = uz * vx - ux * vz, nz = ux * vy - uy * vx;
    area2 += Math.sqrt(nx * nx + ny * ny + nz * nz);
  }

  const signedVolume = signedVolume6 / 6;
  return {
    triangleCount: mesh.triangleCount,
    signedVolume,
    volume: Math.abs(signedVolume),
    surfaceArea: area2 / 2,
    bounds,
    openEdges: countOpenEdges(p),
  };
}

/** Distancia mínima (mm, en cada eje) por debajo de la cual dos vértices se consideran el mismo. */
export const WELD_TOLERANCE = 1e-4;
```

```

/**
 * Lejos del origen el float de 32 bits del STL pierde resolución (a 1000 mm un paso
 * mide ~6e-5 mm), así que la tolerancia crece con el tamaño de las coordenadas.
 */
const WELD_RELATIVE_TOLERANCE = 1e-6;
/** Las celdas de la rejilla miden 16 tolerancias: casi siempre basta con mirar una. */
const WELD_CELL_FACTOR = 16;

/**
 * Suelda vértices casi coincidentes y devuelve un id por vértice (3 números por vértice).
 *
 * Los exportadores no siempre escriben coordenadas idénticas para un vértice compartido
 * (redondeos, conversiones de unidades). Se usa una rejilla espacial; un vértice a
 * menos de `tol` del borde de su celda busca también en la vecina, así dos puntos casi
 * iguales a ambos lados de un borde se unen igualmente.
 */
export function weldVertices(positions: Float32Array, tolerance = WELD_TOLERANCE): { ids: Uint32Array; count: number } {
  const vertexCount = Math.floor(positions.length / 3);
  let maxAbs = 0;
  for (let i = 0; i < vertexCount * 3; i++) {
    const v = Math.abs(positions[i] ?? 0);
    if (v > maxAbs) maxAbs = v;
  }
  const tol = Math.max(tolerance, maxAbs * WELD_RELATIVE_TOLERANCE);
  const cell = tol * WELD_CELL_FACTOR;

  const ids = new Uint32Array(vertexCount);
  // Coordenadas de cada vértice soldado y lista enlazada de los que comparten celda.
  const rep = new Float32Array(vertexCount * 3);
  const nextInCell = new Int32Array(vertexCount);
  const firstInCell = new Map<number, number>();
  const hash = (cx: number, cy: number, cz: number): number =>
    Math.imul(cx, 73856093) ^ Math.imul(cy, 19349663) ^ Math.imul(cz, 83492791);

  let count = 0;
  for (let v = 0; v < vertexCount; v++) {
    const x = positions[v * 3] ?? 0, y = positions[v * 3 + 1] ?? 0, z = positions[v * 3 + 2] ?? 0;
    const cx = Math.floor(x / cell), cy = Math.floor(y / cell), cz = Math.floor(z / cell);
    // Celdas vecinas solo en los ejes en los que el vértice está cerca del borde.
    const x0 = x - cx * cell <= tol ? -1 : 0, x1 = (cx + 1) * cell - x <= tol ? 1 : 0;
    const y0 = y - cy * cell <= tol ? -1 : 0, y1 = (cy + 1) * cell - y <= tol ? 1 : 0;
    const z0 = z - cz * cell <= tol ? -1 : 0, z1 = (cz + 1) * cell - z <= tol ? 1 : 0;

    let found = -1;
    search: for (let dx = x0; dx <= x1; dx++) {
      for (let dy = y0; dy <= y1; dy++) {
        for (let dz = z0; dz <= z1; dz++) {
          // El hash puede colisionar: siempre se comprueban las coordenadas.
          let id = firstInCell.get(hash(cx + dx, cy + dy, cz + dz)) ?? -1;
          while (id >= 0) {
            if (
              Math.abs((rep[id * 3] ?? 0) - x) <= tol &&
              Math.abs((rep[id * 3 + 1] ?? 0) - y) <= tol &&
              Math.abs((rep[id * 3 + 2] ?? 0) - z) <= tol
            ) {
              found = id;
              break search;
            }
            id = nextInCell[id] ?? -1;
          }
        }
      }
    }

    if (found < 0) {
      found = count++;
      rep[found * 3] = x;
      rep[found * 3 + 1] = y;
      rep[found * 3 + 2] = z;
      const key = hash(cx, cy, cz);
      nextInCell[found] = firstInCell.get(key) ?? -1;
      firstInCell.set(key, found);
    }
    ids[v] = found;
  }
  return { ids, count };
}

```

```

}

/**
 * Cuenta las aristas que no comparten exactamente dos triángulos. En una malla
 * cerrada («estanca») el resultado es 0. Antes se sueldan los vértices casi
 * coincidentes (`weldVertices`) para no dar falsos avisos de malla abierta.
 */
export function countOpenEdges(positions: Float32Array, tolerance = WELD_TOLERANCE): number {
  const { ids, count } = weldVertices(positions, tolerance);

  const edges = new Map<number, number>();
  const addEdge = (a: number, b: number): void => {
    if (a === b) return; // arista degenerada (dos vértices soldados en uno)
    const lo = Math.min(a, b), hi = Math.max(a, b);
    const key = lo * count + hi; // exacto: count² < 2⁵³ en cualquier STL que quepa en memoria
    edges.set(key, (edges.get(key) ?? 0) + 1);
  };

  for (let t = 0; t + 2 < ids.length; t += 3) {
    const a = ids[t] ?? 0, b = ids[t + 1] ?? 0, c = ids[t + 2] ?? 0;
    addEdge(a, b);
    addEdge(b, c);
    addEdge(c, a);
  }

  let open = 0;
  for (const count of edges.values()) {
    if (count !== 2) open++;
  }
  return open;
}

export type MeshWarning = 'open-mesh' | 'inverted' | 'too-big' | 'tiny';

/** Por debajo de este tamaño (mm, dimensión mayor) la pieza seguramente se exportó en metros o pulgadas. */
const TINY_PART_MM = 1;

/**
 * Avisos sobre la pieza:
 * - `open-mesh`: volumen ≈ 0 o aristas abiertas: el volumen puede no ser fiable.
 * - `inverted`: volumen con signo negativo (normales hacia dentro).
 * - `too-big`: no cabe en la cama ni girándola 90° sobre Z.
 * - `tiny`: la dimensión mayor mide menos de 1 mm (¿exportada en metros o pulgadas?).
 */
export function meshWarnings(stats: MeshStats, bed: Vec3 = DEFAULT_BED): MeshWarning[] {
  const warnings: MeshWarning[] = [];
  const { size } = stats.bounds;
  const boxVolume = size.x * size.y * size.z;
  const nearZero = Math.abs(stats.signedVolume) <= RELATIVE_VOLUME_EPSILON * Math.max(boxVolume, 1);

  if (nearZero || stats.openEdges > 0) {
    warnings.push('open-mesh');
  } else if (stats.signedVolume < 0) {
    warnings.push('inverted');
  }

  if (!fitsBed(size, bed)) {
    warnings.push('too-big');
  }

  if (Math.max(size.x, size.y, size.z) < TINY_PART_MM) {
    warnings.push('tiny');
  }

  return warnings;
}

/** ¿Cabe la caja en la cama? Se permite girar la pieza 90° sobre el eje Z. */
export function fitsBed(size: Vec3, bed: Vec3): boolean {
  if (size.z > bed.z) return false;
  return (size.x <= bed.x && size.y <= bed.y) || (size.y <= bed.x && size.x <= bed.y);
}

```

src/stl/analyze.ts · 40 líneas

```
import { computeStats } from './geometry';
import { parseStl, StlParseError } from './parse';
import type { Mesh, MeshStats } from './types';

/** Resultado de leer y medir un STL. */
export interface AnalyzedStl {
  readonly mesh: Mesh;
  readonly stats: MeshStats;
}

/** Lee el STL y calcula su geometría. Es lo que hace el worker; también sirve de respaldo sin workers. */
export function analyzeStl(data: ArrayBuffer | Uint8Array): AnalyzedStl {
  const mesh = parseStl(data);
  return { mesh, stats: computeStats(mesh) };
}

export interface AnalyzeRequest {
  readonly id: number;
  readonly buffer: ArrayBuffer;
}

export type AnalyzeResponse =
  | { readonly id: number; readonly ok: true; readonly result: AnalyzedStl }
  | { readonly id: number; readonly ok: false; readonly message: string; readonly parseError: boolean };

/**
 * Atiende un mensaje del worker. Devuelve la respuesta y los buffers que se pueden
 * transferir (sin copiarlos) de vuelta al hilo principal.
 */
export function handleAnalyzeRequest(request: AnalyzeRequest): { response: AnalyzeResponse; transfer: Transferable[] } {
  {
    try {
      const result = analyzeStl(request.buffer);
      return { response: { id: request.id, ok: true, result }, transfer: [result.mesh.positions.buffer as ArrayBuffer] };
    } catch (error) {
      const parseError = error instanceof StlParseError;
      const message = parseError ? error.message : error instanceof Error ? error.message : String(error);
      return { response: { id: request.id, ok: false, message, parseError }, transfer: [] };
    }
  }
}
```

src/stl/analyzer.ts · 89 líneas

```
import { analyzeStl, type AnalyzedStl, type AnalyzeRequest, type AnalyzeResponse } from './analyze';
import { StlParseError } from './parse';

/** Lo mínimo que se usa de un Worker (permite probarlo con un doble en Node). */
export interface WorkerLike {
  postMessage(message: AnalyzeRequest, transfer: Transferable[]): void;
  addEventListener(type: 'message', listener: (event: MessageEvent<AnalyzeResponse>) => void): void;
  addEventListener(type: 'error', listener: (event: Event) => void): void;
  terminate(): void;
}

function createStlWorker(): WorkerLike {
  // Vite detecta esta forma exacta y empaqueta el worker como un archivo aparte.
  return new Worker(new URL('./stl.worker.ts', import.meta.url), { type: 'module' });
}

interface Pending {
  resolve: (result: AnalyzedStl) => void;
  reject: (error: Error) => void;
}

/**
 * Analiza STL en un Web Worker. Si el navegador no puede crear el worker (o este
 * falla al arrancar), hace el trabajo en el hilo principal: más lento, pero funciona.
 */
export class StlAnalyzer {
  private worker: WorkerLike | null = null;
  private workerFailed = false;
  private nextId = 1;
  private readonly pending = new Map<number, Pending>();

  constructor(private readonly createWorker: () => WorkerLike = createStlWorker) {}

  /** Lee y mide el STL. El buffer se transfiere al worker: no se debe reutilizar después. */
  analyze(buffer: ArrayBuffer): Promise<AnalyzedStl> {
    const worker = this.ensureWorker();
    if (!worker) {
      return new Promise((resolve) => resolve(analyzeStl(buffer)));
    }
    const id = this.nextId++;
    return new Promise<AnalyzedStl>((resolve, reject) => {
      this.pending.set(id, { resolve, reject });
      worker.postMessage({ id, buffer }, [buffer]);
    });
  }

  dispose(): void {
    this.worker?.terminate();
    this.worker = null;
    this.rejectAll(new Error('Análisis cancelado.'));
  }

  private ensureWorker(): WorkerLike | null {
    if (this.worker || this.workerFailed) return this.worker;
    try {
      const worker = this.createWorker();
      worker.addEventListener('message', (event) => this.onMessage(event.data));
      worker.addEventListener('error', () => {
        // El worker no ha podido cargarse o ha reventado (p. ej. sin memoria): los
        // siguientes archivos se leerán en el hilo principal.
        this.workerFailed = true;
        this.worker?.terminate();
        this.worker = null;
        this.rejectAll(new Error('El proceso de lectura en segundo plano ha fallado.'));
      });
      this.worker = worker;
    } catch {
      this.workerFailed = true;
    }
    return this.worker;
  }

  private onMessage(response: AnalyzeResponse): void {
    const pending = this.pending.get(response.id);
    if (!pending) return;
    this.pending.delete(response.id);
  }
}
```

```

    if (response.ok) {
        pending.resolve(response.result);
    } else {
        pending.reject(response.parseError ? new StlParseError(response.message) : new Error(response.message));
    }
}

private rejectAll(error: Error): void {
    for (const { reject } of this.pending.values()) reject(error);
    this.pending.clear();
}
}

```

src/stl/stl.worker.ts · 17 líneas

```

// Worker que lee el STL y calcula su geometría fuera del hilo principal, para que la
// interfaz no se congele con archivos de cientos de miles de triángulos.
import { handleAnalyzeRequest, type AnalyzeRequest } from './analyze';

// Tipado mínimo del ámbito del worker (la lib «webworker» choca con la lib «dom» del proyecto).
interface WorkerScope {
    addEventListener(type: 'message', listener: (event: MessageEvent<AnalyzeRequest>) => void): void;
    postMessage(message: unknown, transfer: Transferable[]): void;
}

const scope = self as unknown as WorkerScope;

scope.addEventListener('message', (event) => {
    const { response, transfer } = handleAnalyzeRequest(event.data);
    scope.postMessage(response, transfer);
});

```

src/quote/materials.ts · 24 líneas

```

export type MaterialId = 'PLA' | 'PETG' | 'ABS' | 'TPU';

export interface Material {
    readonly id: MaterialId;
    readonly name: string;
    /** Densidad en g/cm³. */
    readonly density: number;
    /** Precio por defecto en €/kg. */
    readonly defaultPricePerKg: number;
}

export const MATERIALS: Readonly<Record<MaterialId, Material>> = {
    PLA: { id: 'PLA', name: 'PLA', density: 1.24, defaultPricePerKg: 20 },
    PETG: { id: 'PETG', name: 'PETG', density: 1.27, defaultPricePerKg: 24 },
    ABS: { id: 'ABS', name: 'ABS', density: 1.04, defaultPricePerKg: 22 },
    TPU: { id: 'TPU', name: 'TPU', density: 1.21, defaultPricePerKg: 35 },
};

export const MATERIAL_IDS: readonly MaterialId[] = ['PLA', 'PETG', 'ABS', 'TPU'];

export function isMaterialId(value: unknown): value is MaterialId {
    return typeof value === 'string' && (MATERIAL_IDS as readonly string[]).includes(value);
}

```

src/quote/model.ts · 116 líneas

```
import { MATERIALS } from './materials';
import type { QuoteSettings } from './settings';

/** Datos geométricos de la pieza que necesita el presupuesto. */
export interface PartGeometry {
  /** Volumen en mm³. */
  readonly volume: number;
  /** Área de superficie en mm². */
  readonly surfaceArea: number;
}

/** Resultado por copia y total. Volúmenes en mm³, pesos en g, tiempos en h, dinero en €. */
export interface Quote {
  readonly shellVolume: number;
  readonly infillVolume: number;
  /** Volumen impreso por copia. */
  readonly printedVolume: number;
  /** Peso por copia en g. */
  readonly weightGrams: number;
  /** Tiempo estimado por copia en horas (incluye la sobrecarga fija). */
  readonly hoursPerCopy: number;
  readonly copies: number;
  /** Totales para todas las copias. */
  readonly totalWeightGrams: number;
  readonly totalHours: number;
  readonly energyKwh: number;
  readonly materialCost: number;
  readonly energyCost: number;
  /** Coste (material + energía) antes de margen. */
  readonly subtotal: number;
  readonly marginAmount: number;
  readonly total: number;
  readonly totalPerCopy: number;
}

/**
 * Redondea a céntimos. Se pasa antes por 12 cifras significativas para que el ruido
 * de la coma flotante (1,005 guardado como 1,00499999...) no decida el redondeo.
 */
export function roundCents(value: number): number {
  if (!Number.isFinite(value)) return 0;
  return Math.round(Number((value * 100).toFixed(12))) / 100 + 0; // + 0: evita -0
}

/** Grosor de pared en mm: perímetros × ancho de línea. */
export function wallThickness(settings: Pick<QuoteSettings, 'perimeters' | 'lineWidth'>): number {
  return Math.max(0, settings.perimeters) * Math.max(0, settings.lineWidth);
}

/**
 * Volumen impreso por copia (mm³). Modelo simplificado:
 * - cáscara ≈ área × grosor de pared, sin superar el volumen de la pieza;
 * - relleno = relleno% × (volumen - cáscara);
 * - impreso = cáscara + relleno.
 */
export function printedVolume(
  part: PartGeometry,
  settings: Pick<QuoteSettings, 'perimeters' | 'lineWidth' | 'infillPercent'>,
): { shell: number; infill: number; total: number } {
  const volume = Math.max(0, part.volume);
  const shell = Math.min(volume, Math.max(0, part.surfaceArea) * wallThickness(settings));
  const infillRatio = Math.min(100, Math.max(0, settings.infillPercent)) / 100;
  const infill = infillRatio * (volume - shell);
  return { shell, infill, total: shell + infill };
}

/**
 * Presupuesto completo. Función pura: mismos datos, mismo resultado.
 *
 * Los importes se redondean a céntimos línea a línea y el subtotal y el total son la
 * suma de las líneas ya redondeadas, como en una factura: así lo que se ve en el
 * desglose siempre suma el total (antes podía verse 0,08 + 0,00 = 0,09).
 */
export function computeQuote(part: PartGeometry, settings: QuoteSettings): Quote {
  const material = MATERIALS[settings.material];
  const pricePerKg = settings.pricePerKg[settings.material];
```

```

const copies = Math.max(1, Math.round(settings.copies));

const { shell, infill, total: printed } = printedVolume(part, settings);

// mm³ → cm³ (÷1000) × g/cm³ = g
const weightGrams = (printed / 1000) * material.density;

// Tiempo = volumen / caudal + sobrecarga fija (por copia: cada copia es una impresión).
const printSeconds = settings.flowRate > 0 ? printed / settings.flowRate : 0;
const hoursPerCopy = (printSeconds + settings.overheadMinutes * 60) / 3600;

const totalWeightGrams = weightGrams * copies;
const totalHours = hoursPerCopy * copies;
const energyKwh = (settings.powerWatts * totalHours) / 1000;

const materialCost = roundCents((totalWeightGrams / 1000) * pricePerKg);
const energyCost = roundCents(energyKwh * settings.energyPrice);
const subtotal = roundCents(materialCost + energyCost);
const marginAmount = roundCents(subtotal * (settings.marginPercent / 100));
const total = roundCents(subtotal + marginAmount);

return {
  shellVolume: shell,
  infillVolume: infill,
  printedVolume: printed,
  weightGrams,
  hoursPerCopy,
  copies,
  totalWeightGrams,
  totalHours,
  energyKwh,
  materialCost,
  energyCost,
  subtotal,
  marginAmount,
  total,
  totalPerCopy: roundCents(total / copies),
};
}

```

src/quote/settings.ts · 127 líneas

```
import { isMaterialId, MATERIAL_IDS, MATERIALS, type MaterialId } from './materials';

export interface QuoteSettings {
  readonly material: MaterialId;
  /** €/kg de cada material; se recuerda por separado. */
  readonly pricePerKg: Readonly<Record<MaterialId, number>>;
  /** Relleno en % (0-100). */
  readonly infillPercent: number;
  /** Nº de perímetros (paredes). */
  readonly perimeters: number;
  /** Ancho de línea en mm. */
  readonly lineWidth: number;
  /** Caudal volumétrico medio en mm³/s. */
  readonly flowRate: number;
  /** Sobrecarga fija por impresión en minutos (calentar, nivelar, purgar). */
  readonly overheadMinutes: number;
  /** Potencia media de la impresora en W. */
  readonly powerWatts: number;
  /** Precio de la electricidad en €/kWh. */
  readonly energyPrice: number;
  /** Margen sobre el coste en %. */
  readonly marginPercent: number;
  /** Nº de copias. */
  readonly copies: number;
  /** Cama de impresión en mm. */
  readonly bedX: number;
  readonly bedY: number;
  readonly bedZ: number;
}

export const DEFAULT_SETTINGS: QuoteSettings = {
  material: 'PLA',
  pricePerKg: {
    PLA: MATERIALS.PLA.defaultPricePerKg,
    PETG: MATERIALS.PETG.defaultPricePerKg,
    ABS: MATERIALS.ABS.defaultPricePerKg,
    TPU: MATERIALS.TPU.defaultPricePerKg,
  },
  infillPercent: 20,
  perimeters: 2,
  lineWidth: 0.45,
  flowRate: 8,
  overheadMinutes: 5,
  powerWatts: 120,
  energyPrice: 0.15,
  marginPercent: 30,
  copies: 1,
  bedX: 220,
  bedY: 220,
  bedZ: 250,
};

type NumericKey = Exclude<keyof QuoteSettings, 'material' | 'pricePerKg'>;

interface Limit {
  readonly min: number;
  readonly max: number;
  readonly integer?: boolean;
}

/** Rangos válidos de cada ajuste numérico. La interfaz y la carga desde localStorage los respetan. */
export const LIMITS: Readonly<Record<NumericKey | 'pricePerKg', Limit>> = {
  pricePerKg: { min: 0, max: 10000 },
  infillPercent: { min: 0, max: 100 },
  perimeters: { min: 0, max: 20, integer: true },
  lineWidth: { min: 0.1, max: 2 },
  flowRate: { min: 0.1, max: 200 },
  overheadMinutes: { min: 0, max: 600 },
  powerWatts: { min: 0, max: 5000 },
  energyPrice: { min: 0, max: 10 },
  marginPercent: { min: 0, max: 1000 },
  copies: { min: 1, max: 10000, integer: true },
  bedX: { min: 1, max: 5000 },
  bedY: { min: 1, max: 5000 },
  bedZ: { min: 1, max: 5000 },
};
```

```

export function clampToLimit(value: number, limit: Limit): number {
  const rounded = limit.integer ? Math.round(value) : value;
  return Math.min(limit.max, Math.max(limit.min, rounded));
}

/**
 * Convierte cualquier valor (p. ej. lo leído de localStorage) en ajustes válidos:
 * lo que falte o no sea válido toma el valor por defecto y lo numérico se limita a su rango.
 */
export function normalizeSettings(input: unknown): QuoteSettings {
  const source = (typeof input === 'object' && input !== null ? input : {}) as Record<string, unknown>;

  const number = (key: NumericKey): number => {
    const raw = source[key];
    return typeof raw === 'number' && Number.isFinite(raw)
      ? clampToLimit(raw, LIMITS[key])
      : DEFAULT_SETTINGS[key];
  };

  const rawPrices = (typeof source.pricePerKg === 'object' && source.pricePerKg !== null
    ? source.pricePerKg
    : {}) as Record<string, unknown>;
  const pricePerKg = Object.fromEntries(
    MATERIAL_IDS.map((id) => {
      const raw = rawPrices[id];
      const value = typeof raw === 'number' && Number.isFinite(raw)
        ? clampToLimit(raw, LIMITS.pricePerKg)
        : DEFAULT_SETTINGS.pricePerKg[id];
      return [id, value];
    })
  ) as Record<MaterialId, number>;

  return {
    material: isMaterialId(source.material) ? source.material : DEFAULT_SETTINGS.material,
    pricePerKg,
    infillPercent: number('infillPercent'),
    perimeters: number('perimeters'),
    lineWidth: number('lineWidth'),
    flowRate: number('flowRate'),
    overheadMinutes: number('overheadMinutes'),
    powerWatts: number('powerWatts'),
    energyPrice: number('energyPrice'),
    marginPercent: number('marginPercent'),
    copies: number('copies'),
    bedX: number('bedX'),
    bedY: number('bedY'),
    bedZ: number('bedZ'),
  };
}

```

src/quote/format.ts · 54 líneas

```
const LOCALE = 'es-ES';

/**
 * Valor listo para formatear: lo no finito pasa a 0 y lo que se mostraría como «-0,00»
 * (negativos que redondean a cero) también.
 */
function displayable(value: number, decimals: number): number {
  if (!Number.isFinite(value)) return 0;
  return Math.abs(value) < 0.5 * 10 ** -decimals ? 0 : value;
}

const numberFormats = new Map<number, Intl.NumberFormat>();

/** Número con formato es-ES (coma decimal) y un nº fijo de decimales. */
export function formatNumber(value: number, decimals = 2): string {
  let format = numberFormats.get(decimals);
  if (!format) {
    format = new Intl.NumberFormat(LOCALE, {
      minimumFractionDigits: decimals,
      maximumFractionDigits: decimals,
    });
    numberFormats.set(decimals, format);
  }
  return format.format(displayable(value, decimals));
}

const euroFormat = new Intl.NumberFormat(LOCALE, { style: 'currency', currency: 'EUR' });

/** Importe en euros con formato es-ES, p. ej. «12,34 €». */
export function formatEuro(value: number): string {
  return euroFormat.format(displayable(value, 2));
}

/** Duración legible a partir de horas: «45 min», «2 h 05 min». */
export function formatDuration(hours: number): string {
  if (!Number.isFinite(hours) || hours <= 0) return '0 min';
  const totalMinutes = Math.round(hours * 60);
  if (totalMinutes < 1) return '< 1 min';
  const h = Math.floor(totalMinutes / 60);
  const m = totalMinutes % 60;
  if (h === 0) return `${m} min`;
  return `${formatNumber(h, 0)} h ${String(m).padStart(2, '0')} min`;
}

/**
 * Lee un número escrito por una persona: acepta coma o punto decimal y espacios.
 * No acepta separadores de miles. Devuelve NaN si no es un número.
 */
export function parseDecimal(text: string): number {
  const cleaned = text.trim().replace(/\s+/g, '').replace(',', '.');
  if (!/^[-+]?(\d+\.?\d*|\.\d+)\$/.test(cleaned)) return Number.NaN;
  return Number(cleaned);
}
```

src/quote/text.ts · 56 líneas

```
import type { MeshStats } from '../stl/types';
import { formatDuration, formatEuro, formatNumber } from './format';
import { MATERIALS } from './materials';
import type { Quote } from './model';
import type { QuoteSettings } from './settings';

export interface QuoteTextInput {
  readonly fileName: string;
  readonly stats: MeshStats;
  readonly settings: QuoteSettings;
  readonly quote: Quote;
  readonly date?: Date;
}

/** Presupuesto en texto plano, listo para pegar en un correo o mensaje. */
export function buildQuoteText({ fileName, stats, settings, quote, date = new Date() }: QuoteTextInput): string {
  const material = MATERIALS[settings.material];
  const { size } = stats.bounds;
  const dateText = new Intl.DateTimeFormat('es-ES', { dateStyle: 'long' }).format(date);

  const row = (label: string, value: string): string => `${label.padEnd(28, ' ')}${value}`;
  const rule = '-'.repeat(44);

  const lines = [
    'PRESUPUESTO DE IMPRESIÓN 3D',
    `${fileName} · ${dateText}`,
    rule,
    'PIEZA',
    row('Dimensiones (X × Y × Z)', `${formatNumber(size.x, 1)} × ${formatNumber(size.y, 1)} × ${formatNumber(size.z, 1)} mm`),
    row('Volumen', `${formatNumber(stats.volume / 1000, 2)} cm³`),
    row('Superficie', `${formatNumber(stats.surfaceArea / 100, 2)} cm²`),
    rule,
    'AJUSTES',
    row('Material', `${material.name} · ${formatNumber(material.density, 2)} g/cm³ · ${formatEuro(settings.pricePerKg[settings.material])}/kg`),
    row('Relleno', `${formatNumber(settings.infillPercent, 0)} %`),
    row('Perímetros', `${formatNumber(settings.perimeters, 0)} × ${formatNumber(settings.lineWidth, 2)} mm`),
    row('Copias', formatNumber(quote.copies, 0)),
    rule,
    'DESGLOSE',
    row('Volumen impreso por copia', `${formatNumber(quote.printedVolume / 1000, 2)} cm³`),
    row('Peso total', `${formatNumber(quote.totalWeightGrams, 1)} g`),
    row('Tiempo total (estimación)', formatDuration(quote.totalHours)),
    row('Material', formatEuro(quote.materialCost)),
    row('Energía', `${formatEuro(quote.energyCost)} (${formatNumber(quote.energyKwh, 2)} kWh)`),
    row('Subtotal', formatEuro(quote.subtotal)),
    row('Margen (${formatNumber(settings.marginPercent, 0)} %)', formatEuro(quote.marginAmount)),
    rule,
    row('TOTAL', formatEuro(quote.total)),
  ];
  if (quote.copies > 1) {
    lines.push(row('Por copia', formatEuro(quote.totalPerCopy)));
  }
  lines.push('', 'El tiempo es una estimación. Calculado con printquote.');
  return lines.join('\n');
}
```

src/viewer/viewer.ts · 243 líneas

```
import {
  AmbientLight,
  BufferAttribute,
  BufferGeometry,
  Color,
  DirectionalLight,
  EdgesGeometry,
  Float32BufferAttribute,
  Group,
  HemisphereLight,
  LineBasicMaterial,
  LineSegments,
  Mesh,
  MeshStandardMaterial,
  PerspectiveCamera,
  Scene,
  Vector3,
  WebGLRenderer,
} from 'three';
import { OrbitControls } from 'three/addons/controls/OrbitControls.js';
import type { Vec3 } from '../stl/types';

export interface ViewerTheme {
  background: string;
  ink: string;
  line: string;
  accent: string;
}

/** Dirección desde la que mira la cámara al encuadrar (frente-derecha, algo elevada). */
const VIEW_DIRECTION = new Vector3(0.9, -1.35, 0.85).normalize();
const FOV = 35;

/**
 * Visor 3D de la pieza sobre la cama. Trabaja con Z hacia arriba, como los laminadores.
 * Solo renderiza cuando algo cambia (no hay bucle continuo) para no gastar batería.
 */
export class Viewer {
  private readonly renderer: WebGLRenderer;
  private readonly scene = new Scene();
  private readonly camera = new PerspectiveCamera(FOV, 1, 0.1, 20000);
  private readonly controls: OrbitControls;
  private readonly bedGroup = new Group();
  private readonly partGroup = new Group();
  private readonly partMaterial = new MeshStandardMaterial({ roughness: 0.92, metalness: 0, flatShading: true });
  private readonly edgeMaterial = new LineBasicMaterial({ transparent: true, opacity: 0.35 });
  private readonly minorMaterial = new LineBasicMaterial();
  private readonly majorMaterial = new LineBasicMaterial();
  private readonly outlineMaterial = new LineBasicMaterial();
  private readonly resizeObserver: ResizeObserver;
  private bed: Vec3 = { x: 220, y: 220, z: 250 };
  private partSize: Vec3 | null = null;
  private frame = 0;

  constructor(private readonly container: HTMLElement, theme: ViewerTheme) {
    this.renderer = new WebGLRenderer({ antialias: true });
    this.renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
    this.renderer.domElement.setAttribute('aria-hidden', 'true');
    container.appendChild(this.renderer.domElement);

    this.camera.up.set(0, 0, 1);
    this.controls = new OrbitControls(this.camera, this.renderer.domElement);
    // Inercia al soltar, salvo que se haya pedido reducir el movimiento.
    this.controls.enableDamping = !window.matchMedia('(prefers-reduced-motion: reduce)').matches;
    this.controls.dampingFactor = 0.12;
    // Teclado (con el foco en el visor): flechas para desplazar, Mayús + flechas para girar.
    this.controls.keyRotateSpeed = 15;
    this.controls.listenToKeyEvents(container);
    this.controls.addEventListener('change', () => this.requestRender());

    this.scene.add(new HemisphereLight(0xffffffff, 0x8a8580, 1.6));
    this.scene.add(new AmbientLight(0xffffffff, 0.35));
    const key = new DirectionalLight(0xffffffff, 1.6);
    key.position.set(-1.2, -2, 3);
    this.scene.add(key);
    const rim = new DirectionalLight(0xffffffff, 0.5);
```

```

    rim.position.set(2, 3, 1);
    this.scene.add(rim);

    this.scene.add(this.bedGroup, this.partGroup);
    this.setTheme(theme);
    this.buildBed();

    this.resizeObserver = new ResizeObserver(() => this.resize());
    this.resizeObserver.observe(container);
    this.resize();
    this.resetCamera();
}

setTheme(theme: ViewerTheme): void {
    this.scene.background = new Color(theme.background);
    this.partMaterial.color.set(theme.accent);
    this.edgeMaterial.color.set(theme.ink);
    this.minorMaterial.color.set(theme.line);
    this.majorMaterial.color.set(new Color(theme.line).lerp(new Color(theme.ink), 0.25));
    this.outlineMaterial.color.set(theme.ink);
    this.requestRender();
}

/** Cambia el tamaño de la cama (mm) y redibuja la rejilla. */
setBed(bed: Vec3): void {
    if (bed.x === this.bed.x && bed.y === this.bed.y && bed.z === this.bed.z) return;
    this.bed = bed;
    this.buildBed();
    this.requestRender();
}

/**
 * Muestra una malla (9 números por triángulo, mm). Se centra en la cama y se
 * apoya sobre ella (Z mínima = 0) solo para verla; los cálculos usan el original.
 */
setPart(positions: Float32Array): void {
    this.clearPart();
    const geometry = new BufferGeometry();
    geometry.setAttribute('position', new BufferAttribute(positions.slice(), 3));
    geometry.computeBoundingBox();
    const box = geometry.boundingBox;
    if (!box) return;
    const center = box.getCenter(new Vector3());
    geometry.translate(-center.x, -center.y, -box.min.z);
    geometry.computeVertexNormals();

    const mesh = new Mesh(geometry, this.partMaterial);
    this.partGroup.add(mesh);

    // Aristas marcadas (ángulo > 30°) para leer bien la forma, como en un plano.
    if (positions.length / 9 <= 400_000) {
        this.partGroup.add(new LineSegments(new EdgesGeometry(geometry, 30), this.edgeMaterial));
    }

    const size = box.getSize(new Vector3());
    this.partSize = { x: size.x, y: size.y, z: size.z };
    this.resetCamera();
}

clearPart(): void {
    for (const child of [...this.partGroup.children]) {
        this.partGroup.remove(child);
        if (child instanceof Mesh || child instanceof LineSegments) child.geometry.dispose();
    }
    this.partSize = null;
    this.requestRender();
}

/** Encuadra la pieza (o la cama si no hay pieza) desde la vista por defecto. */
resetCamera(): void {
    const target = new Vector3();
    let radius: number;
    if (this.partSize) {
        const s = this.partSize;
        target.set(0, 0, s.z / 2);
        radius = Math.hypot(s.x, s.y, s.z) / 2;
    } else {

```

```

    radius = Math.hypot(this.bed.x, this.bed.y) / 2;
  }
  radius = Math.max(radius, 1);

  const vFov = (FOV * Math.PI) / 180;
  const hFov = 2 * Math.atan(Math.tan(vFov / 2) * this.camera.aspect);
  const distance = (radius / Math.sin(Math.min(vFov, hFov) / 2)) * 1.3;

  this.camera.near = Math.max(0.1, distance / 1000);
  this.camera.far = distance * 50;
  this.camera.position.copy(target).addScaledVector(VIEW_DIRECTION, distance);
  this.camera.updateProjectionMatrix();
  this.controls.target.copy(target);
  this.controls.minDistance = radius * 0.2;
  this.controls.maxDistance = distance * 8;
  this.controls.update();
  this.requestRender();
}

/** Imagen PNG de la vista actual (para la hoja impresa). */
snapshot(): string {
  this.renderer.render(this.scene, this.camera);
  return this.renderer.domElement.toDataURL('image/png');
}

dispose(): void {
  cancelAnimationFrame(this.frame);
  this.resizeObserver.disconnect();
  this.controls.dispose();
  this.clearPart();
  this.renderer.dispose();
  this.renderer.domElement.remove();
}

private requestRender(): void {
  if (this.frame) return;
  this.frame = requestAnimationFrame(() => {
    this.frame = 0;
    // Con amortiguación, update() devuelve true mientras la cámara siga moviéndose.
    const moving = this.controls.update();
    this.renderer.render(this.scene, this.camera);
    if (moving) this.requestRender();
  });
}

private resize(): void {
  const width = Math.max(1, this.container.clientWidth);
  const height = Math.max(1, this.container.clientHeight);
  this.renderer.setSize(width, height, false);
  this.camera.aspect = width / height;
  this.camera.updateProjectionMatrix();
  this.requestRender();
}

/** Rejilla de la cama: línea fina cada 10 mm, marcada cada 50 mm y contorno. */
private buildBed(): void {
  for (const child of [...this.bedGroup.children]) {
    this.bedGroup.remove(child);
    if (child instanceof LineSegments) child.geometry.dispose();
  }
  const { x: w, y: d } = this.bed;
  const x0 = -w / 2, y0 = -d / 2;
  const minor: number[] = [];
  const major: number[] = [];

  for (let x = 10; x < w; x += 10) {
    (x % 50 === 0 ? major : minor).push(x0 + x, y0, 0, x0 + x, y0 + d, 0);
  }
  for (let y = 10; y < d; y += 10) {
    (y % 50 === 0 ? major : minor).push(x0, y0 + y, 0, x0 + w, y0 + y, 0);
  }
  const outline = [
    x0, y0, 0, x0 + w, y0, 0,
    x0 + w, y0, 0, x0 + w, y0 + d, 0,
    x0 + w, y0 + d, 0, x0, y0 + d, 0,
    x0, y0 + d, 0, x0, y0, 0,
  ];
}

```

```

    const lines = (points: number[], material: LineBasicMaterial): LineSegments => {
      const geometry = new BufferGeometry();
      geometry.setAttribute('position', new Float32BufferAttribute(points, 3));
      return new LineSegments(geometry, material);
    };
    // Un poco por debajo de Z = 0 para que la rejilla no «pelee» con la base de la pieza.
    this.bedGroup.position.z = -0.05;
    this.bedGroup.add(lines(minor, this.minorMaterial), lines(major, this.majorMaterial), lines(outline, this.
outlineMaterial));
  }
}

```

src/viewer/webgl.ts · 13 líneas

```

/**
 * ¿Puede este navegador crear un contexto WebGL? Vive aparte del visor para poder
 * comprobarlo sin descargar three.js.
 */
export function supportsWebGL(): boolean {
  try {
    const canvas = document.createElement('canvas');
    return Boolean(canvas.getContext('webgl2') ?? canvas.getContext('webgl'));
  } catch {
    return false;
  }
}

```

src/ui/app.ts · 446 líneas

```
import { formatDuration, formatEuro, formatNumber } from '../quote/format';
import { isMaterialId, MATERIALS } from '../quote/materials';
import { computeQuote, type Quote } from '../quote/model';
import { DEFAULT_SETTINGS, LIMITS, normalizeSettings, type QuoteSettings } from '../quote/settings';
import { buildQuoteText } from '../quote/text';
import { StlAnalyzer } from '../stl/analyzer';
import { meshWarnings, type MeshWarning } from '../stl/geometry';
import { StlParseError } from '../stl/parse';
import type { Mesh, MeshStats, Vec3 } from '../stl/types';
import type { Viewer, ViewerTheme } from '../viewer/viewer';
import { supportsWebGL } from '../viewer/webgl';
import { NumberField } from './number-field';
import { renderPrintSheet } from './print-sheet';
import { clearSettings, loadSettings, saveSettings } from './storage';

const SAMPLE_FILE = 'soporte-movil.stl';
const MAX_FILE_BYTES = 300 * 1024 * 1024;

interface LoadedPart {
  readonly fileName: string;
  readonly mesh: Mesh;
  readonly stats: MeshStats;
}

function byId<T extends HTMLElement> = HTMLElement(id: string): T {
  const node = document.getElementById(id);
  if (!node) throw new Error(`Falta el elemento #${id} en index.html`);
  return node as T;
}

function readTheme(): ViewerTheme {
  const style = getComputedStyle(document.documentElement);
  const token = (name: string, fallback: string): string => style.getPropertyValue(name).trim() || fallback;
  return {
    background: token('--stage', '#eeece7'),
    ink: token('--ink', '#111111'),
    line: token('--line', '#d9d6cf'),
    accent: token('--accent', '#ff5a1f'),
  };
}

function warningText(warning: MeshWarning, size: Vec3, bed: Vec3): string {
  switch (warning) {
    case 'open-mesh':
      return 'La malla parece abierta o con huecos: el volumen, el peso y el precio pueden no ser fiables. Repárala en tu laminador o editor 3D.';
    case 'inverted':
      return 'Las normales parecen invertidas (volumen negativo). Se usa el valor absoluto, pero conviene revisar la malla.';
    case 'tiny':
      return `La pieza mide solo ${formatNumber(Math.max(size.x, size.y, size.z), 3)} mm en su lado mayor. printquote asume que el STL está en milímetros: si se exportó en metros o pulgadas, cambia las unidades al exportar.`;
    case 'too-big':
      return `La pieza (${formatNumber(size.x, 1)} × ${formatNumber(size.y, 1)} × ${formatNumber(size.z, 1)} mm) no cabe en la cama de ${formatNumber(bed.x, 0)} × ${formatNumber(bed.y, 0)} × ${formatNumber(bed.z, 0)} mm, ni siquiera girándola.`;
  }
}

async function copyText(text: string): Promise<boolean> {
  try {
    await navigator.clipboard.writeText(text);
    return true;
  } catch {
    // Respaldo para contextos sin la API del portapapeles.
    const area = document.createElement('text-area');
    area.value = text;
    area.setAttribute('readonly', '');
    area.style.position = 'fixed';
    area.style.opacity = '0';
    document.body.append(area);
    area.select();
    const ok = document.execCommand('copy');
    area.remove();
    return ok;
  }
}
```

```

}

/** Arranca la aplicación: enlaza controles, visor, arrastrar y soltar y almacenamiento. */
export function startApp(): void {
  const stage = byId<'viewer'>.parentElement as HTMLElement;
  const notice = byId<'notice'>;
  const live = byId<'live-status'>;
  const loading = byId<'stage-loading'>;
  const fileInput = byId<HTMLInputElement><'file-input'>;
  const resetCameraButton = byId<HTMLButtonElement><'reset-camera'>;
  const copyButton = byId<HTMLButtonElement><'copy-button'>;
  const printButton = byId<HTMLButtonElement><'print-button'>;
  const infillRange = byId<HTMLInputElement><'in-infill-range'>;
  const materialRadios = Array.from(document.querySelectorAll<HTMLInputElement><'input[name="material"]'>));
  const out = (id: string): HTMLElement => byId<'out-${id}'>;
  /** Solo toca el DOM si el texto cambia: el total es una región viva y no debe repetirse. */
  const setOut = (id: string, text: string): void => {
    const node = out(id);
    if (node.textContent !== text) node.textContent = text;
  };

  let settings = loadSettings();
  let part: LoadedPart | null = null;
  let quote: Quote | null = null;
  let viewer: Viewer | null = null;

  const analyzer = new StlAnalyzer();
  /** Cada carga recibe un número; si llega otra antes de terminar, el resultado viejo se descarta. */
  let loadTicket = 0;

  // — Visor —
  // three.js (~500 kB) se descarga aparte, después de pintar la interfaz: el panel
  // y el presupuesto funcionan desde el primer momento aunque el visor tarde.
  const noViewer = (): void =>
    showNotice<'Sin vista 3D', 'Tu navegador no ha podido iniciar WebGL. El presupuesto funciona igualmente.'>;
  if (supportsWebGL()) {
    import<'../viewer/viewer'>
      .then(({ Viewer }) => {
        const container = byId<'viewer'>;
        viewer = new Viewer(container, readTheme());
        // El visor admite teclado (lo gestiona OrbitControls): se hace enfocable y se explica.
        container.tabIndex = 0;
        container.setAttribute<'role', 'application'>;
        container.setAttribute<'aria-roledescription', 'visor 3D'>;
        container.setAttribute(
          'aria-label',
          'Vista 3D de la pieza. Flechas: desplazar. Mayúsculas más flechas: girar. «Restablecer vista» la vuelve a
          encuadrar.',
        );
        if (part) viewer.setPart(part.mesh.positions);
        render();
      })
      .catch(() => {
        viewer = null;
        noViewer();
      });
  } else {
    noViewer();
  }
  window.matchMedia<'(prefers-color-scheme: dark)'>.addEventListener<'change', () => viewer?.setTheme(readTheme())>;

  // — Campos —
  const bed = (): Vec3 => ({ x: settings.bedX, y: settings.bedY, z: settings.bedZ });

  const update = (patch: Partial<QuoteSettings>): void => {
    settings = normalizeSettings({ ...settings, ...patch });
    saveSettings(settings);
    render();
  };

  type FieldKey = Exclude<keyof QuoteSettings, 'material' | 'pricePerKg'>;
  const fieldSpec: ReadonlyArray<readonly [inputId: string, errorId: string, key: FieldKey | 'price', decimals:
  number, step: number]> = [
    ['in-price', 'err-price', 'price', 2, 1],
    ['in-infill', 'err-infill', 'infillPercent', 1, 5],
    ['in-perimeters', 'err-perimeters', 'perimeters', 0, 1],
    ['in-linewidth', 'err-linewidth', 'linewidth', 2, 0.05],
  ]

```

```

    ['in-flow', 'err-flow', 'flowRate', 1, 1],
    ['in-bedx', 'err-bed', 'bedX', 0, 10],
    ['in-bedy', 'err-bed', 'bedY', 0, 10],
    ['in-bedz', 'err-bed', 'bedZ', 0, 10],
    ['in-power', 'err-power', 'powerWatts', 0, 10],
    ['in-energy', 'err-energy', 'energyPrice', 3, 0.01],
    ['in-margin', 'err-margin', 'marginPercent', 1, 5],
    ['in-copies', 'err-copies', 'copies', 0, 1],
  ];

const fields = new Map<FieldKey | 'price', NumberField>();
for (const [inputId, errorId, key, decimals, step] of fieldSpec) {
  const limit = key === 'price' ? LIMITS.pricePerKg : LIMITS[key];
  const field = new NumberField({
    input: byId<HTMLInputElement>(inputId),
    error: byId(errorId),
    min: limit.min,
    max: limit.max,
    integer: limit.integer ?? false,
    decimals,
    step,
    onValue: (value) => {
      if (key === 'price') {
        update({ pricePerKg: { ...settings.pricePerKg, [settings.material]: value } });
      } else {
        update({ [key]: value });
      }
    },
  });
  fields.set(key, field);
}

const syncForm = (): void => {
  for (const [key, field] of fields) {
    field.set(key === 'price' ? settings.pricePerKg[settings.material] : settings[key]);
  }
  for (const radio of materialRadios) radio.checked = radio.value === settings.material;
  infillRange.value = String(settings.infillPercent);
};

for (const radio of materialRadios) {
  radio.addEventListener('change', () => {
    if (radio.checked && isMaterialId(radio.value)) {
      update({ material: radio.value });
      fields.get('price')?.set(settings.pricePerKg[settings.material]);
    }
  });
}

infillRange.addEventListener('input', () => {
  const value = Number(infillRange.value);
  fields.get('infillPercent')?.set(value);
  update({ infillPercent: value });
});

byId('reset-settings').addEventListener('click', () => {
  clearSettings();
  settings = DEFAULT_SETTINGS;
  syncForm();
  render();
  announce('Ajustes restablecidos a los valores por defecto.');
```

```

});

// — Render —
function render(): void {
  const b = bed();
  viewer?.setBed(b);
  byId('stage-legend').textContent = `Rejilla 10 mm · Cama ${formatNumber(b.x, 0)} × ${formatNumber(b.y, 0)} × ${formatNumber(b.z, 0)} mm`;
  infillRange.value = String(settings.infillPercent);
  infillRange.setAttribute('aria-valuetext', `${formatNumber(settings.infillPercent, 0)} %`);

  const hasPart = part !== null;
  stage.classList.toggle('has-part', hasPart);
  resetCameraButton.disabled = !hasPart || !viewer;
  copyButton.disabled = !hasPart;
  printButton.disabled = !hasPart;
}

```

```

const warningsList = byId('warnings');
if (!part) {
  quote = null;
  for (const id of ['volume', 'area', 'size', 'triangles', 'printed', 'weight', 'time', 'material', 'energy',
'subtotal', 'margin', 'total']) {
    setOut(id, '-');
  }
  setOut('kwh', '');
  setOut('margin-pct', '');
  setOut('summary', 'Carga una pieza para calcular el presupuesto.');
```

byId('stage-dims').textContent = '';

warningsList.replaceChildren();

return;

}

const { stats } = part;

const size = stats.bounds.size;

quote = computeQuote(stats, settings);

setOut('volume', formatNumber(stats.volume / 1000, 2));

setOut('area', formatNumber(stats.surfaceArea / 100, 2));

setOut('size', `\${formatNumber(size.x, 1)} × \${formatNumber(size.y, 1)} × \${formatNumber(size.z, 1)}`);

setOut('triangles', formatNumber(stats.triangleCount, 0));

const dims = byId('stage-dims');

dims.replaceChildren(

...(['x', 'y', 'z'] as const).map((axis) => {

const span = document.createElement('span');

const label = document.createElement('b');

label.textContent = axis.toUpperCase();

span.append(label, `\${formatNumber(size[axis], 1)}`);

return span;

}),

'mm',

);

// La lista es una región viva: solo se reescribe si los avisos cambian, para que el

// lector de pantalla no los repita cada vez que se toca un ajuste.

const warningTexts = meshWarnings(stats, b).map((warning) => warningText(warning, size, b));

const currentTexts = Array.from(warningsList.children, (li) => li.textContent ?? '');

if (warningTexts.join('|') !== currentTexts.join('|')) {

warningsList.replaceChildren(

...warningTexts.map((text) => {

const li = document.createElement('li');

li.textContent = text;

return li;

}),

);

}

setOut('printed', formatNumber(quote.printedVolume / 1000, 2));

setOut('weight', formatNumber(quote.totalWeightGrams, 1));

setOut('weight-sub', quote.copies > 1 ? `total · \${formatNumber(quote.copies, 0)} copias` : 'total');

setOut('time', formatDuration(quote.totalHours));

setOut('material', formatNumber(quote.materialCost, 2));

setOut('energy', formatNumber(quote.energyCost, 2));

setOut('kwh', `\${formatNumber(quote.energyKwh, 2)} kWh`);

setOut('subtotal', formatNumber(quote.subtotal, 2));

setOut('margin-pct', `\${formatNumber(settings.marginPercent, 0)} %`);

setOut('margin', formatNumber(quote.marginAmount, 2));

setOut('total', formatEuro(quote.total));

const copiesText = quote.copies === 1 ? '1 copia' : `\${formatNumber(quote.copies, 0)} copias · \${formatEuro(quote.totalPerCopy)}/copia`;

out('summary').textContent =

`\${copiesText} · \${MATERIALS[settings.material].name} \${formatNumber(settings.infillPercent, 0)} % · ` +

`\${formatDuration(quote.totalHours)} (estimación)`;

}

// — Carga de archivos —

function showNotice(title: string, message: string): void {

const heading = document.createElement('p');

heading.className = 'notice-title';

heading.textContent = title;

const body = document.createElement('p');

body.textContent = message;

notice.replaceChildren(heading, body);

```

    notice.setAttribute('role', 'alert');
    notice.hidden = false;
}

function hideNotice(): void {
    notice.hidden = true;
    notice.removeAttribute('role');
    notice.replaceChildren();
}

function announce(message: string): void {
    live.textContent = '';
    window.setTimeout(() => (live.textContent = message), 50);
}

async function loadBuffer(fileName: string, getBuffer: () => Promise<ArrayBuffer>): Promise<void> {
    const ticket = ++loadTicket;
    hideNotice();
    stage.classList.add('is-loading');
    stage.setAttribute('aria-busy', 'true');
    loading.textContent = `Leyendo «${fileName}»...`;
    loading.hidden = false;
    announce(`Leyendo ${fileName}...`);
    try {
        const buffer = await getBuffer();
        // El parseo y la geometría van en un Web Worker: la interfaz sigue respondiendo.
        const { mesh, stats } = await analyzer.analyze(buffer);
        if (ticket !== loadTicket) return; // ya se ha pedido otro archivo
        part = { fileName, mesh, stats };
        viewer?.setPart(mesh.positions);

        byId('file-name').textContent = fileName;
        byId('file-detail').textContent =
            `STL ${mesh.format === 'binary' ? 'binario' : 'ASCII'} · ${formatNumber(mesh.triangleCount, 0)} triángulos`;
        document.title = `${fileName} · printquote`;
        render();
        // El total se anuncia solo: su contenedor es una región viva.
        announce(`Pieza cargada: ${fileName}.`);
    } catch (error) {
        if (ticket !== loadTicket) return;
        const message = error instanceof StlParseError
            ? error.message
            : 'Ha ocurrido un error inesperado al leer el archivo. Prueba a exportarlo de nuevo como STL.';
        showNotice(`No se ha podido leer «${fileName}»`, message);
    } finally {
        if (ticket === loadTicket) {
            stage.classList.remove('is-loading');
            stage.removeAttribute('aria-busy');
            loading.hidden = true;
        }
    }
}

function loadFile(file: File): void {
    if (file.size > MAX_FILE_BYTES) {
        showNotice(`«${file.name}» es demasiado grande`, 'El límite es de 300 MB para no bloquear el navegador.');
```

```

const overlay = byId('drop-overlay');
let dragDepth = 0;
const hasFiles = (event: DragEvent): boolean => Array.from(event.dataTransfer?.types ?? []).includes('Files');

window.addEventListener('dragenter', (event) => {
  if (!hasFiles(event)) return;
  event.preventDefault();
  dragDepth++;
  overlay.hidden = false;
});
window.addEventListener('dragover', (event) => {
  if (!hasFiles(event)) return;
  event.preventDefault();
  if (event.dataTransfer) event.dataTransfer.dropEffect = 'copy';
});
window.addEventListener('dragleave', (event) => {
  if (!hasFiles(event)) return;
  dragDepth = Math.max(0, dragDepth - 1);
  if (dragDepth === 0) overlay.hidden = true;
});
window.addEventListener('drop', (event) => {
  if (!hasFiles(event)) return;
  event.preventDefault();
  dragDepth = 0;
  overlay.hidden = true;
  const file = event.dataTransfer?.files[0];
  if (file) loadFile(file);
});

// — Acciones —
resetCameraButton.addEventListener('click', () => viewer?.resetCamera());

const copyLabel = copyButton.textContent;
let copyLabelTimer = 0;
copyButton.addEventListener('click', () => {
  if (!part || !quote) return;
  const text = buildQuoteText({ fileName: part.fileName, stats: part.stats, settings, quote });
  void copyText(text).then((ok) => {
    // Etiqueta original guardada fuera: con dos clics seguidos se quedaba en «Copiado».
    copyButton.textContent = ok ? 'Copiado' : 'No se pudo copiar';
    announce(ok ? 'Presupuesto copiado al portapapeles.' : 'No se ha podido copiar el presupuesto.');
```

window.clearTimeout(copyLabelTimer);
 copyLabelTimer = window.setTimeout(() => (copyButton.textContent = copyLabel), 1800);
 });
});

const fillPrintSheet = (): void => {
 if (!part || !quote) return;
 renderPrintSheet(byId('print-sheet'), {
 fileName: part.fileName,
 format: part.mesh.format,
 stats: part.stats,
 settings,
 quote,
 image: viewer ? viewer.snapshot() : null,
 });
};
window.addEventListener('beforeprint', fillPrintSheet);
printButton.addEventListener('click', () => {
 fillPrintSheet();
 window.print();
});

syncForm();
render();

// Enlace directo a la demo con la pieza cargada: .../printquote/#ejemplo
if (window.location.hash === '#ejemplo') loadSample();
}

src/ui/number-field.ts · 123 líneas

```
import { parseDecimal } from '../quote/format';

export interface NumberFieldOptions {
  readonly input: HTMLInputElement;
  readonly error: HTMLElement | null;
  readonly min: number;
  readonly max: number;
  readonly integer?: boolean;
  /** Máximo de decimales al mostrar el valor. */
  readonly decimals: number;
  /** Paso de las flechas ↑/↓ (Mayús: ×10). */
  readonly step: number;
  readonly onValue: (value: number) => void;
}

const inputFormats = new Map<number, Intl.NumberFormat>();

/** Valor para un campo editable: coma decimal, sin separador de miles, sin ceros sobrantes. */
export function formatInputValue(value: number, decimals: number): string {
  let format = inputFormats.get(decimals);
  if (!format) {
    format = new Intl.NumberFormat('es-ES', { maximumFractionDigits: decimals, useGrouping: false });
    inputFormats.set(decimals, format);
  }
  return format.format(value);
}

/**
 * Campo numérico de texto que acepta coma o punto decimal, valida en vivo,
 * muestra el error junto al campo y admite ↑/↓ para ajustar el valor.
 */
export class NumberField {
  private value = 0;

  constructor(private readonly options: NumberFieldOptions) {
    const { input } = options;
    input.addEventListener('input', () => this.onInput());
    input.addEventListener('change', () => this.onCommit());
    input.addEventListener('blur', () => this.onCommit());
    input.addEventListener('keydown', (event) => this.onKeyDown(event));
  }

  /** Pone un valor desde fuera (sin disparar `onValue`). */
  set(value: number): void {
    this.value = value;
    this.options.input.value = formatInputValue(value, this.options.decimals);
    this.setError(null);
  }

  private parse(): number {
    const value = parseDecimal(this.options.input.value);
    return this.options.integer && Number.isFinite(value) && !Number.isInteger(value) ? Number.NaN : value;
  }

  private inRange(value: number): boolean {
    return Number.isFinite(value) && value >= this.options.min && value <= this.options.max;
  }

  private rangeMessage(): string {
    const { min, max, integer, decimals } = this.options;
    const kind = integer ? 'un número entero' : 'un número';
    return `Introduce ${kind} entre ${formatInputValue(min, decimals)} y ${formatInputValue(max, decimals)}.`;
  }

  private onInput(): void {
    const value = this.parse();
    if (this.inRange(value)) {
      this.setError(null);
      this.commit(value);
    } else {
      this.setError(this.rangeMessage());
    }
  }

  /** Al salir del campo: se limita al rango o, si no es un número, se vuelve al último valor válido. */
  private onCommit(): void {

```

```

    const value = this.parse();
    if (Number.isFinite(value)) {
        const clamped = Math.min(this.options.max, Math.max(this.options.min, value));
        this.commit(clamped);
    }
    this.set(this.value);
}

private onKeyDown(event: KeyboardEvent): void {
    if (event.key !== 'ArrowUp' && event.key !== 'ArrowDown') return;
    event.preventDefault();
    const current = this.inRange(this.parse()) ? this.parse() : this.value;
    const step = this.options.step * (event.shiftKey ? 10 : 1) * (event.key === 'ArrowUp' ? 1 : -1);
    // Redondeo para evitar arrastrar errores de coma flotante (0,1 + 0,2...).
    const factor = 10 ** this.options.decimals;
    const next = Math.round((current + step) * factor) / factor;
    const clamped = Math.min(this.options.max, Math.max(this.options.min, next));
    this.commit(clamped);
    this.set(clamped);
}

private commit(value: number): void {
    if (value === this.value) return;
    this.value = value;
    this.options.onValue(value);
}

private setError(message: string | null): void {
    const { input, error } = this.options;
    if (message) {
        input.setAttribute('aria-invalid', 'true');
        if (error) {
            // El párrafo de error es una región viva siempre presente (vacía = invisible):
            // así los lectores de pantalla anuncian el mensaje en cuanto aparece.
            if (error.textContent !== message) error.textContent = message;
            error.dataset.owner = input.id;
        }
    } else {
        input.removeAttribute('aria-invalid');
        if (error && error.dataset.owner === input.id) {
            error.textContent = '';
            delete error.dataset.owner;
        }
    }
}
}

```

src/ui/print-sheet.ts · 114 líneas

```
import { formatDuration, formatEuro, formatNumber } from '../quote/format';
import { MATERIALS } from '../quote/materials';
import type { Quote } from '../quote/model';
import type { QuoteSettings } from '../quote/settings';
import type { MeshStats, StlFormat } from '../stl/types';

export interface PrintSheetData {
  readonly fileName: string;
  readonly format: StlFormat;
  readonly stats: MeshStats;
  readonly settings: QuoteSettings;
  readonly quote: Quote;
  readonly image: string | null;
}

type Row = readonly [label: string, value: string];

function el<K extends keyof HTMLElementTagNameMap>(tag: K, className?: string, text?: string): HTMLElementTagNameMap[K] {
  const node = document.createElement(tag);
  if (className) node.className = className;
  if (text !== undefined) node.textContent = text;
  return node;
}

function table(caption: string, rows: readonly Row[], totalRow?: Row): HTMLTableElement {
  const t = el('table', 'ps-table');
  t.append(el('caption', undefined, caption));
  const body = el('tbody');
  const addRow = ([label, value]: Row, className?: string): void => {
    const tr = el('tr', className);
    const th = el('th', undefined, label);
    th.scope = 'row';
    tr.append(th, el('td', undefined, value));
    body.append(tr);
  };
  rows.forEach((row) => addRow(row));
  if (totalRow) addRow(totalRow, 'ps-total');
  t.append(body);
  return t;
}

/** Rellena la hoja de impresión. Se construye con nodos DOM (nunca innerHTML) porque el nombre del archivo lo pone la persona usuaria. */
export function renderPrintSheet(container: HTMLElement, data: PrintSheetData): void {
  const { fileName, format, stats, settings, quote, image } = data;
  const material = MATERIALS[settings.material];
  const size = stats.bounds.size;
  const date = new Intl.DateTimeFormat('es-ES', { dateStyle: 'long' }).format(new Date());

  const head = el('header', 'ps-head');
  const meta = el('div', 'ps-meta');
  meta.append(el('div', undefined, date), el('div', undefined, 'bertmarti.github.io/printquote'));
  head.append(el('div', 'ps-brand', 'printquote'), meta);

  const title = el('h1', 'ps-title', 'Presupuesto de impresión 3D');
  const file = el(
    'p',
    'ps-file',
    `${fileName} · STL ${format === 'binary' ? 'binario' : 'ASCII'} · ${formatNumber(stats.triangleCount, 0)} triángulos`,
  );

  const nodes: HTMLElement[] = [head, title, file];

  if (image) {
    const figure = el('figure', 'ps-figure');
    const img = el('img');
    img.src = image;
    img.alt = `Vista 3D de ${fileName}`;
    figure.append(img);
    nodes.push(figure);
  }

  const grid = el('div', 'ps-grid');
  grid.append(
```

```

    table('Pieza', [
      ['Dimensiones (X × Y × Z)', `${formatNumber(size.x, 1)} × ${formatNumber(size.y, 1)} × ${formatNumber(size.z,
1)} mm`],
      ['Volumen', `${formatNumber(stats.volume / 1000, 2)} cm³`],
      ['Superficie', `${formatNumber(stats.surfaceArea / 100, 2)} cm²`],
      ['Volumen impreso por copia', `${formatNumber(quote.printedVolume / 1000, 2)} cm³`],
      ['Peso por copia', `${formatNumber(quote.weightGrams, 1)} g`],
    ]),
    table('Ajustes', [
      ['Material', `${material.name} · ${formatNumber(material.density, 2)} g/cm³`],
      ['Precio del material', `${formatEuro(settings.pricePerKg[settings.material])/kg`],
      ['Relleno · perímetros', `${formatNumber(settings.infillPercent, 0)} % · ${formatNumber(settings.perimeters,
0)} × ${formatNumber(settings.lineWidth, 2)} mm`],
      ['Caudal volumétrico', `${formatNumber(settings.flowRate, 1)} mm³/s`],
      ['Potencia · electricidad', `${formatNumber(settings.powerWatts, 0)} W · ${formatNumber(settings.energyPrice,
3)} €/kWh`],
    ]),
  );
  nodes.push(grid);

  const breakdown: Row[] = [
    ['Copias', formatNumber(quote.copies, 0)],
    ['Peso total', `${formatNumber(quote.totalWeightGrams, 1)} g`],
    ['Tiempo total (estimación)', formatDuration(quote.totalHours)],
    ['Material', formatEuro(quote.materialCost)],
    ['Energía', `${formatNumber(quote.energyKwh, 2)} kWh · ${formatEuro(quote.energyCost)}`],
    ['Subtotal', formatEuro(quote.subtotal)],
    ['Margen (${formatNumber(settings.marginPercent, 0)} %)`, formatEuro(quote.marginAmount)],
  ];
  if (quote.copies > 1) breakdown.push(['Precio por copia', formatEuro(quote.totalPerCopy)]);
  nodes.push(table('Desglose', breakdown, ['Total', formatEuro(quote.total)]));

  nodes.push(
    el(
      'p',
      'ps-note',
      'El tiempo de impresión es una estimación basada en un caudal volumétrico medio; el laminador dará la cifra
real. ' +
      'El archivo se ha procesado en el navegador y no se ha enviado a ningún servidor.',
    ),
  );

  container.replaceChildren(...nodes);
}

```

src/ui/storage.ts · 30 líneas

```

import { DEFAULT_SETTINGS, normalizeSettings, type QuoteSettings } from '../quote/settings';

const STORAGE_KEY = 'printquote:ajustes:v1';

/** Lee los ajustes guardados. Si no hay, están corruptos o el navegador bloquea el almacenamiento, usa los de por
defecto. */
export function loadSettings(): QuoteSettings {
  try {
    const raw = window.localStorage.getItem(STORAGE_KEY);
    return normalizeSettings(raw ? (JSON.parse(raw) as unknown) : undefined);
  } catch {
    return DEFAULT_SETTINGS;
  }
}

export function saveSettings(settings: QuoteSettings): void {
  try {
    window.localStorage.setItem(STORAGE_KEY, JSON.stringify(settings));
  } catch {
    // Modo privado o almacenamiento lleno: la app sigue funcionando sin recordar ajustes.
  }
}

export function clearSettings(): void {
  try {
    window.localStorage.removeItem(STORAGE_KEY);
  } catch {
    // Ídem.
  }
}

```


scripts/generate-sample.mjs · 132 líneas

```
// Genera la pieza de ejemplo de printquote: un soporte de móvil low-poly.
// Diseño original de este proyecto (licencia MIT, como el resto del repositorio).
//
// Uso: npm run sample → public/samples/soporte-movil.stl (STL binario, mm)
//
// La pieza es la extrusión en X de un perfil dibujado en el plano Y-Z:
// una pata delantera con labio para apoyar el móvil, un respaldo inclinado ~65°
// y una pata trasera, unidas por arriba y con un arco abierto por debajo para
// ahorrar material. Al ser una extrusión de un polígono simple, la malla es
// cerrada y con normales hacia fuera.

import { mkdirSync, writeFileSync } from 'node:fs';
import { dirname, resolve } from 'node:path';
import { fileURLToPath } from 'node:url';

const WIDTH = 70; // mm en X

/** Perfil en sentido antihorario, puntos [y, z] en mm (y = fondo, z = altura). */
const PROFILE = [
  [0, 0],
  [24.4, 0], // fin de la pata delantera
  [50.4, 60], // arco interior (lado delantero)
  [61, 60], // techo del arco
  [77.9, 0], // arco interior (lado trasero)
  [90, 0],
  [90, 5],
  [84, 8], // chaflán trasero
  [62, 86], // cara trasera del respaldo
  [55, 88], // cumbre
  [22, 12], // cara de apoyo del móvil (~65°)
  [12, 12], // ranura donde descansa el canto del móvil
  [12, 22], // labio
  [6, 22],
  [0, 14], // chaflán delantero
];

function signedArea(poly) {
  let a = 0;
  for (let i = 0; i < poly.length; i++) {
    const [x1, y1] = poly[i];
    const [x2, y2] = poly[(i + 1) % poly.length];
    a += x1 * y2 - x2 * y1;
  }
  return a / 2;
}

function cross(o, a, b) {
  return (a[0] - o[0]) * (b[1] - o[1]) - (a[1] - o[1]) * (b[0] - o[0]);
}

function insideTriangle(p, a, b, c) {
  return cross(a, b, p) >= 0 && cross(b, c, p) >= 0 && cross(c, a, p) >= 0;
}

/** Triangulación por recorte de orejas de un polígono simple antihorario. Devuelve índices. */
function triangulate(poly) {
  const remaining = poly.map((_, i) => i);
  const triangles = [];
  let guard = 0;
  while (remaining.length > 3) {
    if (guard++ > 10000) throw new Error('No se pudo triangular el perfil');
    for (let k = 0; k < remaining.length; k++) {
      const ip = remaining[(k - 1 + remaining.length) % remaining.length];
      const ic = remaining[k];
      const inx = remaining[(k + 1) % remaining.length];
      const [a, b, c] = [poly[ip], poly[ic], poly[inx]];
      if (cross(a, b, c) <= 0) continue; // vértice cóncavo
      const blocked = remaining.some(
        (j) => j !== ip && j !== ic && j !== inx && insideTriangle(poly[j], a, b, c),
      );
      if (blocked) continue;
      triangles.push([ip, ic, inx]);
      remaining.splice(k, 1);
      break;
    }
  }
}
```

```

    triangles.push([remaining[0], remaining[1], remaining[2]]);
    return triangles;
}

function buildTriangles() {
  if (signedArea(PROFILE) <= 0) throw new Error('El perfil debe ser antihorario');
  const at = (x, [y, z]) => [x, y, z];
  const tris = [];

  // Tapas: en x = WIDTH la normal es +X (orden antihorario en Y-Z); en x = 0, invertida.
  for (const [a, b, c] of triangulate(PROFILE)) {
    tris.push([at(WIDTH, PROFILE[a]), at(WIDTH, PROFILE[b]), at(WIDTH, PROFILE[c])]);
    tris.push([at(0, PROFILE[a]), at(0, PROFILE[c]), at(0, PROFILE[b])]);
  }

  // Paredes: un quad por arista del perfil, con la normal hacia fuera.
  for (let i = 0; i < PROFILE.length; i++) {
    const p = PROFILE[i];
    const q = PROFILE[(i + 1) % PROFILE.length];
    const a0 = at(0, p), a1 = at(0, q), b1 = at(WIDTH, q), b0 = at(WIDTH, p);
    tris.push([a0, a1, b1], [a0, b1, b0]);
  }
  return tris;
}

function normal([a, b, c]) {
  const u = [b[0] - a[0], b[1] - a[1], b[2] - a[2]];
  const v = [c[0] - a[0], c[1] - a[1], c[2] - a[2]];
  const n = [u[1] * v[2] - u[2] * v[1], u[2] * v[0] - u[0] * v[2], u[0] * v[1] - u[1] * v[0]];
  const len = Math.hypot(...n) || 1;
  return n.map((value) => value / len);
}

function toBinaryStl(tris, header) {
  const buffer = Buffer.alloc(84 + tris.length * 50);
  buffer.write(header.padEnd(80, ' ').slice(0, 80), 0, 'ascii');
  buffer.writeUInt32LE(tris.length, 80);
  tris.forEach((tri, i) => {
    let offset = 84 + i * 50;
    for (const value of [...normal(tri), ...tri.flat()]) {
      buffer.writeFloatLE(value, offset);
      offset += 4;
    }
    buffer.writeUInt16LE(0, offset);
  });
  return buffer;
}

const tris = buildTriangles();
const out = resolve(dirname(fileURLToPath(import.meta.url)), '../public/samples/soporte-movil.stl');
mkdirSync(dirname(out), { recursive: true });
writeFileSync(out, toBinaryStl(tris, 'printquote sample - soporte de movil low-poly (MIT)'));

const volume = signedArea(PROFILE) * WIDTH;
console.log(`${out}\n${tris.length} triángulos · volumen ${(volume / 1000).toFixed(2)} cm³`);

```

`.github/workflows/ci.yml` · 42 líneas

```
name: CI

on:
  pull_request:
  push:
    branches: [main]

permissions:
  contents: read

concurrency:
  group: ci-{{ github.ref }}
  cancel-in-progress: true

jobs:
  check:
    name: Lint, tests y build ({{ matrix.os }})
    runs-on: {{ matrix.os }}
    strategy:
      fail-fast: false
      matrix:
        os: [ubuntu-latest, windows-latest]
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm

      - name: Instalar dependencias
        run: npm ci

      - name: Lint
        run: npm run lint

      - name: Tests
        run: npm test

      - name: Build
        run: npm run build
```

.github/workflows/deploy.yml · 53 líneas

```
name: Despliegue en GitHub Pages

on:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: pages
  cancel-in-progress: false

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm

      - name: Instalar dependencias
        run: npm ci

      - name: Tests
        run: npm test

      - name: Build (base /printquote/)
        run: npm run build

      - uses: actions/configure-pages@v5

      - uses: actions/upload-pages-artifact@v3
        with:
          path: dist

  deploy:
    needs: build
    runs-on: ubuntu-latest
    environment:
      name: github-pages
      url: ${ steps.deployment.outputs.page_url }
    steps:
      - name: Publicar en Pages
        id: deployment
        uses: actions/deploy-pages@v4
```